Fpt University

TRƯỜNG ĐẠI HỌC FPT

CAPSTONE PROJECT REPORT

Report 4 – Software Design Document

Automatic Attendance System Based on Real-Time Location and Face Recognition

INSTRUCTOR: BÙI THỊ LOAN

CLASS: SEP490_G16

Team Members:

1.	Trần Long Khánh	HE176617
2.	Nguyễn Đình Dũng	HE176518
3.	Hà Việt Hiếu	HE176256
4.	Nguyễn Văn Đạt	HE176009
5.	Kiều Thị Ngọc Dung	HE171203

Table of Contents

I. Record of Changes.....7
II. Software Design Document..... 8
1. System Design.....8
1.1 System Architecture.....8
1.2 Package Diagram.....15
2. Database Design.....23
2.1 SQL Database.....23
2.2 NoSQL Database.....31
3. Detailed Design.....34
3.1. Attendance Calculation.....34
3.2. FaceID verification.....37
4. Other Detailed Design.....40
4.1 AI pipeline.....40
4.2 Model use:.....43
4.3 Model Description.....44

List of Figures

Figure No.	Title	Page
Figure 1	Figure 1.1.1 Architecture Diagram	10
Figure 2	Figure 1.1.2.1 Mobile App	10
Figure 3	Figure 1.1.2.2 ASP.NET	11
Figure 4	Figure 1.1.2.3 Redis	12
Figure 5	Figure 1.1.2.4 PostgreSQL with pgvector	12
Figure 6	Figure 1.1.2.5 RabbitMQ	
Figure 7	Figure 1.1.2.6 MartenDB	
Figure 8	Figure 1.1.2.7 Docker	
Figure 9	Figure 1.1.2.8 MediaPipe	
Figure 10	Figure 1.1.2.9 TensorFlow Lite	
Figure 11	Figure 1.2.1 Frontend Package Diagram	
Figure 12	Figure 1.2.2 Mobile Package Diagram	
Figure 13	Figure 1.2.3 Backend Package Diagram	
Figure 14	Figure 2.1 SQL Database Design	
Figure 15	Figure 2.2 NoSQL Database Design	
Figure 16	Figure 3.1.1 Attendance Calculation class diagram	
Figure 17	Figure 3.1.2 Attendance Calculation sequence diagram	
Figure 18	Figure 3.2.1 FaceID verification class diagram	
Figure 19	Figure 3.2.2 FaceID verification sequence diagram	

Figure 20	Figure 4.1 FaceID AI pipeline diagram	
Figure 21	Figure 4.1.2 FaceID verification activity diagram	
Figure 22	Figure 4.3.1.1 Anti-spoofting architecture	
Figure 23	Figure 4.3.2.1 iTracker - Gaze estimation architecture	
Figure 24	Figure 4.3.3.1 FaceNet model structure	
Figure 25	Figure 4.3.3.1 FaceNet model triplet loss	

Table 1 - List of Figures

List of Tables

Table No.	Title	Page
Table 1	Table 1 - List of Figures	
Table 2	Table 2 - List of Tables	
Table 3	Table 3 - Definitions & Acronyms	
Table 4	Table 4 Package Descriptions	
Table 5	Table 1.2.1 Package Descriptions	
Table 6	Table 1.2.3 Package Descriptions	
Table 7	Table 2.1.1 User Description	
Table 8	Table 2.1.2 Account Description	
Table 9	Table 2.1.3 User Request Description	
Table 10	Table 2.1.4 Device Description	
Table 11	Table 2.1.5 Attribute Definition Description	
Table 12	Table 2.1.6 Option Description	
Table 13	Table 2.1.7 Setting Description	
Table 14	Table 2.1.8 User Attribute Description	
Table 15	Table 2.1.9 Course Description	
Table 16	Table 2.1.10 Class Section Description	
Table 17	Table 2.1.11 Enrollment Description	
Table 18	Table 2.1.12 Room Description	
Table 19	Table 2.1.13 Schedule Description	

Table 20	Table 2.1.14 Session Description	
Table 21	Table 2.1.15 Round Description	
Table 22	Table 2.1.16 Attendance Record Description	
Table 23	Table 2.1.17 Face Embedding Description	
Table 24	Table 2.1.18 FaceI Id Verify Request Description	
Table 25	Table 2.2.1 ScheduleWhitelist Description	
Table 26	Table 2.2.2 Scan Log Description	
Table 27	Table 2.2.3 Round Track Description	
Table 28	Table 2.2.4 Student Track Description	
Table 29	Table 2.2.5 Notification Description	
Table 30	Table 4.3.1.1 Anti-spoofting model description	

Table 2 - List of Tables

Definitions & Acronyms

No	Acronyms & Abbreviations	Definition
1	AAS	Automatic Attendance System
2	ERD	Entity Relationship Diagram
3	API	Application Programming Interface
4	URL	Uniform Resource Locator
5	HTTP	Hypertext Transfer Protocol
6	REST	Representational State Transfer
7	JWT	JSON Web Token
8	CQRS	Command Query Responsibility Segregation

Table 3 - Definitions & Acronyms

I. Record of Changes

Date	A* M, D	In charge	Change Description
12/6/2025	A	DatNV	Add System Design
13/6/2025	A	DatNV	Add Component Description, Package Diagram Overall
14/6/2025	A	DatNV	Add Database Design
16/6/2025	A	DatNV	Add Mock Detailed Design/Class - Sequence Diagram
22/6/2025	M	DatNV	Update Database Description
25/6/2025	M	DatNV	Update Database Description
28/6/2025	A	DatNV	Add Detailed Design/Class Diagram (1, 2)
29/6/2025	A	DatNV	Add Detailed Design/Class Diagram (3 -> 14)
12/8/2025	M	DatNV	Update Database Design
12/8/2025	M	DatNV	Update Package Diagram
16/8/2025	M	DatNV	Update Package Diagram
16/8/2025	M	DatNV	Update Class Diagram
17/8/2025	M	DatNV	Update Component Description, Database Design
17/8/2025	A	HieuHV	Add Detailed Design of FaceID
18/8/2025	M	DatNV	Update ERD Diagram, Architecture Diagram
18/8/2025	A	HieuHV	Add AI pipeline diagram and models description

**A - Added M - Modified D - Deleted*

II. Software Design Document

1. System Design

1.1 System Architecture

1.1.1 Architecture Diagram

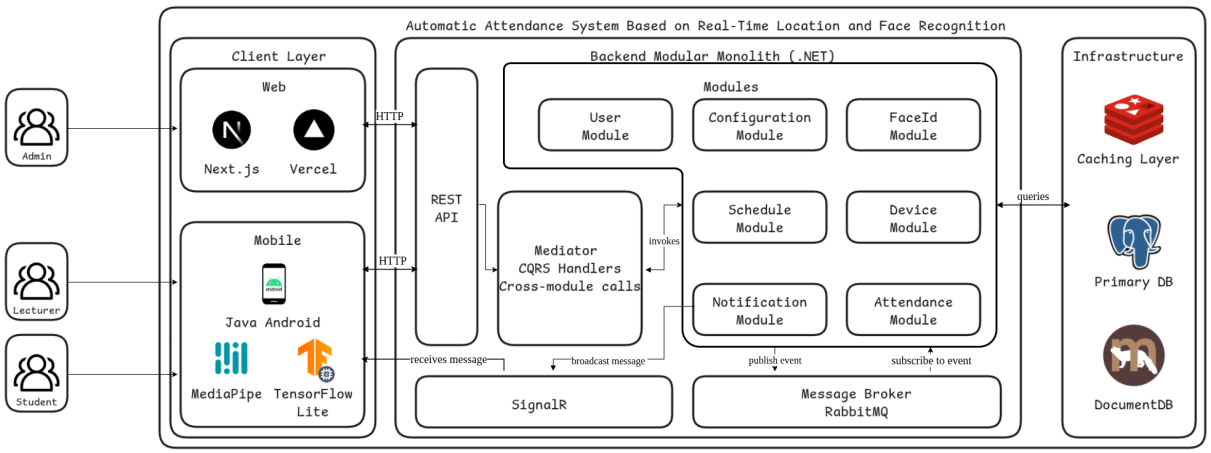


Figure 1.1.1 Architecture Diagram

1.1.2 System Architecture Explanation

The Automatic Attendance System comprises a client-side component with dual interfaces: a web application built with Next.js and a mobile application developed in Java Android, alongside a server-side component constructed using ASP.NET Core following a modular monolithic architecture. Communication between clients and servers utilizes REST APIs with comprehensive authentication and authorization through ASP.NET Identity and JWT token-based security. The backend employs the CQRS pattern through a Mediator implementation across multiple business modules including User Management, Attendance Management, Schedule Management, Device Management, FaceID verification, and Notification Services.

For data persistence, the system utilizes a hybrid database approach: PostgreSQL with pgvector extension serves as the primary relational database for structured data and facial embeddings storage, while MartenDB provides document-based storage for attendance tracking data. Redis acts as a caching layer, and RabbitMQ serves as a message broker for asynchronous communication. The mobile application leverages on-device machine learning through MediaPipe and TensorFlow Lite for face recognition processing, enabling offline-first attendance capture with automatic synchronization. Docker containerization ensures consistent deployment across different environments.

1.1.2 Component Description

1.1.2.1 Mobile App

The Android application, developed with **Java**, serves as the primary end-user interface. It facilitates user interactions and data collection, connecting with the backend via **REST APIs** to manage all system functionalities. This component is crucial for displaying system information and executing user-driven tasks.



Figure 1.1.2.1 Mobile App

Features and Benefits:

- **User Interface (UI):** Provides a comprehensive interface for displaying real-time data and system states.
- **REST API Integration:** Ensures seamless communication with the ASP.NET backend for data exchange.
- **Data Collection:** Gathers essential information from the end-user environment to be processed by the server.
- **Cross-Platform Compatibility:** Runs natively on Android, ensuring an optimized user experience.

1.1.2.2 ASP.NET

ASP.NET is the core backend framework, handling all server-side logic and managing system workflows. It coordinates with various other components like Redis, PostgreSQL, and RabbitMQ to ensure robust task execution and service integration.



Figure 1.1.2.2 ASP.NET

Features and Benefits:

- **Centralized Logic:** Manages all business logic and orchestrates data flow between different modules.
- **Component Integration:** Acts as a central hub, integrating with Redis for caching, PostgreSQL for data persistence, and RabbitMQ for asynchronous messaging.
- **Scalability:** Handles high volumes of user requests and complex task executions efficiently.
- **Security:** Provides a secure environment for all server-side operations and API handling.

1.1.2.3 Redis

Redis is an in-memory data store used as a high-speed cache. It is strategically implemented to enhance performance by storing temporary data, reducing the load on the primary database for frequent queries.



Figure 1.1.2.3 Redis

Features and Benefits:

- **Performance Optimization:** Drastically reduces latency for high-volume read operations.
- **Load Reduction:** Offloads traffic from the PostgreSQL database, improving its performance and stability.
- **Real-time Data Access:** Enables swift retrieval of temporary and frequently accessed data.
- **Flexible Data Structures:** Supports various data structures, including strings, hashes, and lists, for efficient caching.

1.1.2.4 PostgreSQL with pgvector

PostgreSQL serves as the primary relational database for the system. It is responsible for the persistent storage of all critical data, including user profiles, system configurations, and historical records. By leveraging the **pgvector extension**, it gains powerful capabilities for handling vector embeddings, which is essential for advanced features like **FaceID**. Its robust features ensure data integrity and reliable data management.



Figure 1.1.2.4 PostgreSQL with pgvector

Features and Benefits:

- **Vector Search for FaceID:** The **pgvector** extension allows for efficient storage and querying of high-dimensional vectors. This is used to store facial recognition embeddings, enabling fast and accurate matches for the **FaceID** feature.
- **Data Integrity:** Enforces ACID properties, guaranteeing reliable transactions.
- **Complex Querying:** Supports advanced SQL queries for detailed data analysis and reporting.
- **Reliability:** Provides built-in features for data backups and disaster recovery.
- **Extensibility:** Allows for the use of extensions like **pgvector** to add new data types and functions, expanding its core capabilities.

1.1.2.5. RabbitMQ

RabbitMQ is a message broker that facilitates asynchronous communication between different components of the ASP.NET backend. It ensures reliable message delivery and decouples services, which is essential for handling high-load scenarios and maintaining system responsiveness.



Figure 1.1.2.5 RabbitMQ

Features and Benefits:

- **Asynchronous Processing:** Enables the backend to handle long-running or resource-intensive tasks without blocking the main thread.
- **Decoupling:** Modules can communicate without knowing each other's details, simplifying system architecture.
- **Load Distribution:** Distributes tasks evenly across different worker processes, ensuring scalability.
- **Guaranteed Delivery:** Provides robust mechanisms to ensure that messages are processed even if a service fails.

1.1.2.6. MartenDB

MartenDB is utilized as a **document database** and event store built on top of **PostgreSQL**. It is particularly effective for managing complex, non-relational data structures like those used for tracking attendance and user activity. By using MartenDB, the project leverages the reliability of PostgreSQL while gaining the flexibility and performance of a document-oriented model.



Figure 1.1.2.6 MartenDB

Features and Benefits:

- **Document-Oriented Storage:** Stores data as JSON documents, which is ideal for flexible schemas like those for attendance records and user tracks.
- **Event Sourcing:** Provides built-in support for event sourcing, enabling a reliable audit trail of all changes.
- **Native PostgreSQL Integration:** Uses PostgreSQL's robust features like transactions and backup, ensuring data durability and integrity.
- **Performance:** Optimized for high-volume writes and complex queries on document data, complementing the relational data in the main PostgreSQL instance.

1.1.2.7. Docker

Docker is used to containerize the application's components, ensuring consistency across all development, testing, and deployment environments. By isolating each service—like the ASP.NET backend, the PostgreSQL database, and RabbitMQ—into its own container, Docker simplifies the deployment process, improves scalability, and reduces environmental inconsistencies.



Figure 1.1.2.7 Docker

Features and Benefits:

- **Environment Consistency:** Guarantees that the application runs the same way everywhere, from a developer's local machine to production servers.
- **Scalability:** Enables rapid scaling of services by running multiple instances of a container.
- **Dependency Isolation:** Each container includes its own dependencies, preventing conflicts between services.
- **Efficiency:** Reduces resource consumption and speeds up deployment cycles by using a lightweight, layered file system.

1.1.2.8. MediaPipe

MediaPipe is used for implementing machine learning solutions for computer vision and perception tasks. It is essential for processing video and image data in real-time to enable features such as face detection, hand tracking, and object recognition within the application.



Figure 1.1.2.8 MediaPipe

Features and Benefits:

- **Real-Time Processing:** Delivers low-latency performance for live video analysis on mobile devices.
- **Cross-Platform:** Supports deployment on various platforms, including Android, iOS, and desktop.
- **Integrated Solutions:** Provides pre-built pipelines for common tasks, simplifying development.
- **Robustness:** Performs reliably in diverse environmental conditions, handling complex vision-based tasks.

1.1.2.9. TensorFlow Lite

TensorFlow Lite is an optimized version of TensorFlow specifically designed for running machine learning models on mobile and embedded devices. It is integrated to execute models efficiently on the end-user devices, ensuring high performance without requiring a constant network connection.



Figure 1.1.2.9 TensorFlow Lite

Features and Benefits:

- **On-Device Machine Learning:** Executes models directly on the mobile app, reducing latency and reliance on cloud services.
- **Model Optimization:** Compresses and optimizes models for size and speed, ideal for resource-constrained devices.
- **Offline Capability:** Enables machine learning functionalities even when the device is not connected to the internet.
- **Hardware Acceleration:** Utilizes mobile device hardware to speed up model inference.

1.2 Package Diagram

1.2.1. Frontend Package Diagram

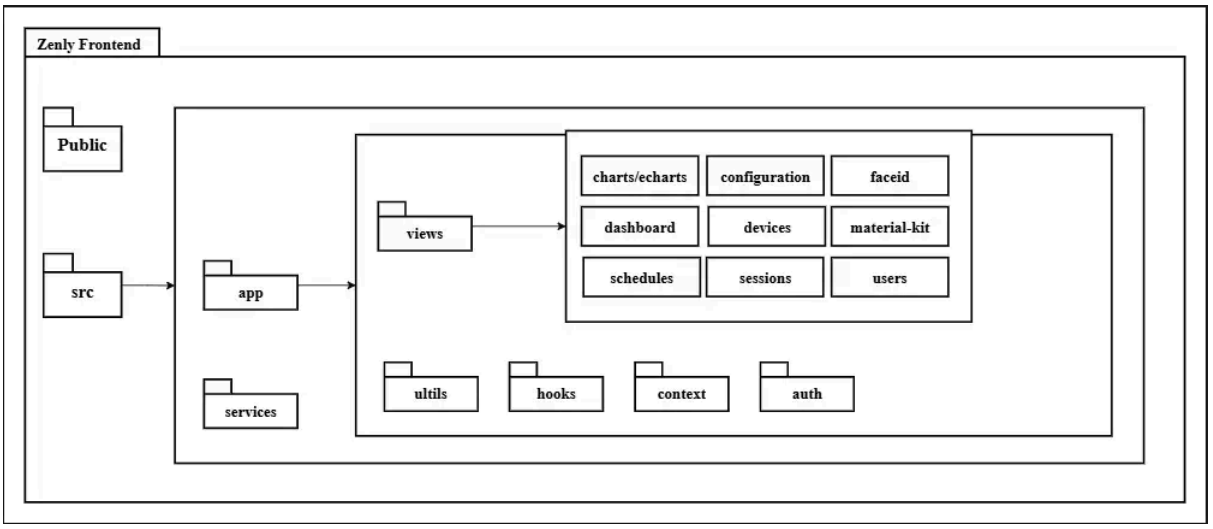


Figure 1.2.1 Frontend Package Diagram

Package Descriptions

No	Package	Description
01	public	Contains static assets such as images, icons, and configuration files that are directly served by the browser.
02	src	Main source folder of the frontend application, containing all React code, business logic, and supporting modules.

03	app		Application entry point. Defines global layout, routes, theme, and sets up core providers. Acts as the root orchestrator of the frontend.
04	views	charts/echarts	Provides reusable chart components for analytics and monitoring. Built on ECharts, it supports line, bar, pie, and custom visualizations.
05		configuration	UI module for managing system configurations and runtime settings. Allows administrators to update attributes, options, and preferences.
06		faceid	Interface for face recognition enrollment and verification. Integrates with backend FaceId services for biometric identity checks.
07		dashboard	Main overview page summarizing key system metrics and activities. Offers quick access to core modules with widgets and statistics.
08		devices	Manages device registration, status monitoring, and details. Provides interfaces for authentication, metadata, and whitelist control.
09		material-kit	Contains shared UI components, layouts, and style guides. Ensures consistency in look and feel across all modules.
10		schedules	Manages views for courses, timetables, and room allocations. Allows users to view, create, and update scheduling information.

11		sessions	Displays attendance sessions and tracking information. Provides tools for scanning, logging, and reviewing attendance records.
12		users	Handles user profile management, roles, and permissions. Includes views for registration, authentication, and account settings.
13	services		API service layer that encapsulates HTTP requests and communication with the backend. Provides reusable client-side abstractions for data access.
14	utils		Utility functions and helpers shared across the frontend application.
15	hooks		Custom React hooks that encapsulate reusable stateful logic.
16	context		React Context providers and consumers for managing global application state and cross-cutting concerns.
17	auth		Authentication and authorization handling. Manages login flows, token storage, role-based access, and session state.

Table 1.2.1 Package Descriptions

1.2.2. Mobile Package Diagram

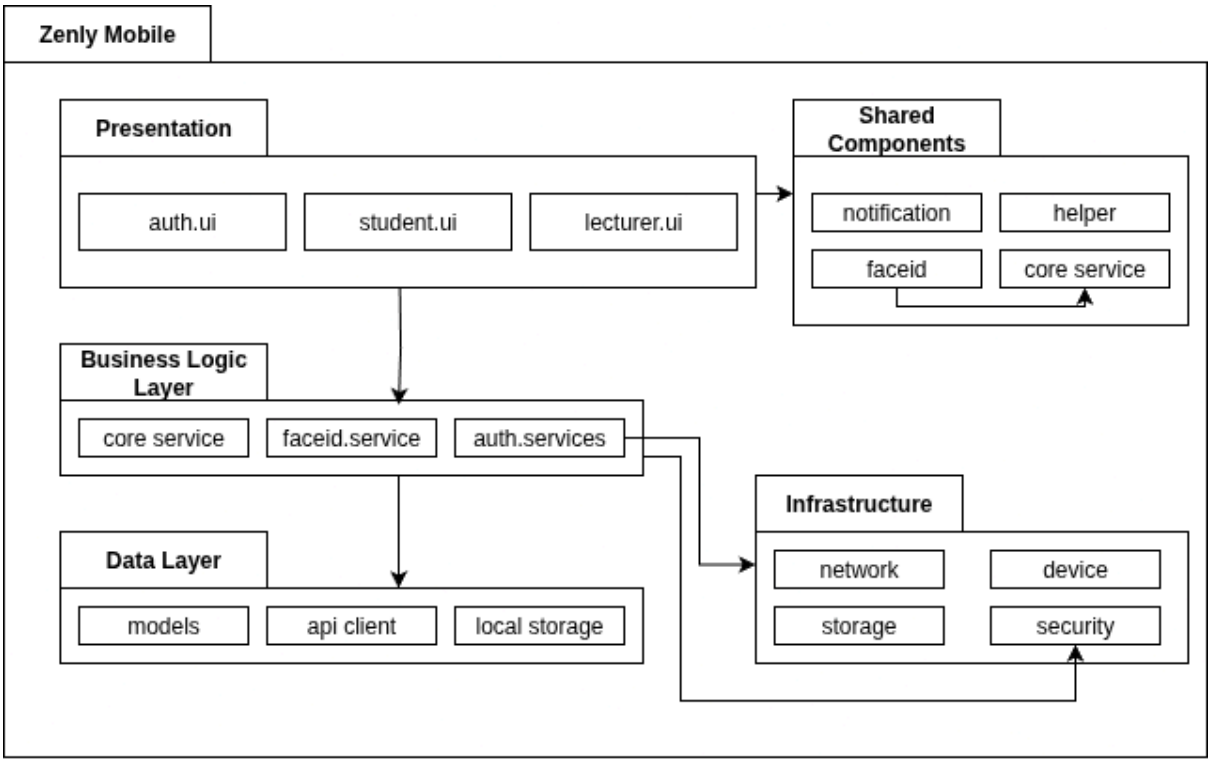


Figure 1.2.2 Mobile Package Diagram

Package Descriptions

No	Package		Description
01	Presentation	auth.ui	Provides user authentication screens such as login, registration, and password reset. Ensures secure user onboarding with backend integration.
02		student.ui	UI for students including dashboards, schedules, and attendance views. Designed for simplicity and daily use in managing learning activities.
03		lecturer.ui	UI for lecturers to manage courses, attendance, and class notifications. Focuses on oversight and control features with quick access tools.

04	Business Logic Layer	core service	Provides core business functions reusable across the app. Handles shared operations like validation, mapping, and state coordination.
05		faceid.service	Implements biometric recognition logic for secure login and attendance. Encapsulates communication with device cameras and backend verification.
06		auth.services	Handles authentication workflows, token management, and role-based session state. Ensures smooth user access across multiple roles.
07	Data Layer	models	Defines data entities and domain models used across the app. Provides a consistent structure for API responses and local persistence.
08		api client	Responsible for HTTP communication with backend APIs. Manages request building, response parsing, and error handling.
09		local storage	Stores user sessions, cached data, and offline records. Enables offline-first experience with synchronization to backend.
10	Shared Components	notification	Provides reusable UI components for push and in-app notifications. Supports alerts, reminders, and contextual system messages.
11		helper	Utility functions for formatting, calculations, and shared logic. Reduces code duplication across presentation and services.

12		faceid	Reusable biometric UI and helper functions for multiple flows. Connects face recognition features across modules.
13		core service	Shared foundational services such as configuration, error handling, and context providers. Accessible by both business logic and UI.
14	Infrastructure	network	Encapsulates network setup including API base configuration, interceptors, and error retries. Ensures reliable and secure communication.
15		storage	Abstracts access to persistent storage. Provides unified APIs for data read/write.
16		device	Handles interactions with device hardware. Provides abstraction over native APIs for cross-platform use.
		security	Implements encryption, token protection, and secure session handling. Ensures sensitive data is safely stored and transmitted.

Table 1.2.1 Package Descriptions

1.2.3. Backend Package Diagram

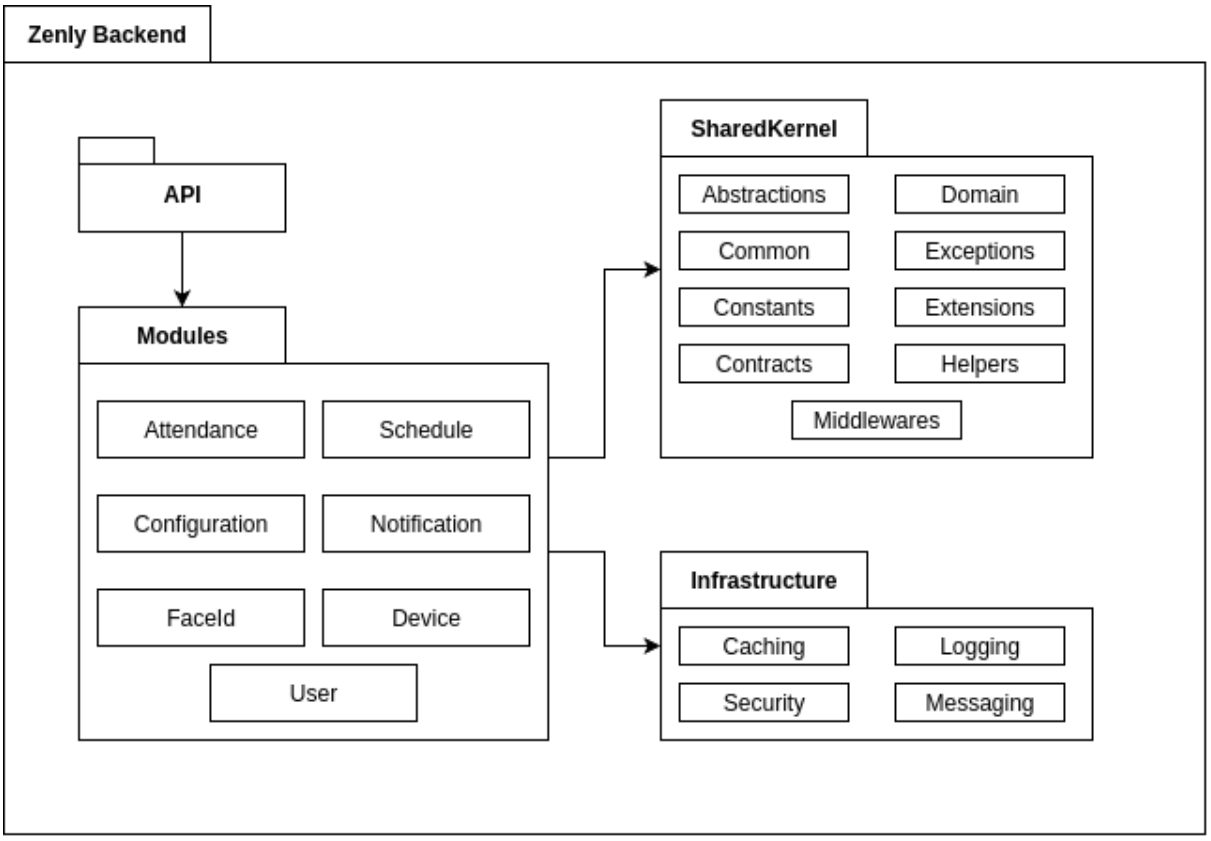


Figure 1.2.3 Backend Package Diagram

Package Descriptions

No	Package	Description
01	API	Main application entry point exposing HTTP APIs. Configures startup, middleware, DI, and acts as the public gateway to all modules.
02	Infrastructure	Provides cross-cutting infrastructure such as caching, logging, persistence, messaging, and security implementations used by modules.
03	SharedKernel	Contains shared domain primitives, constants, exceptions, contracts, and reusable abstractions for all modules.

04	Modules	AttendanceManagement	Handles attendance domain including sessions, rounds, scan logs, and attendance records. Provides calculation logic, APIs, and persistence.
05		ConfigurationManagement	Manages system configuration such as attributes, settings, and options with scope-based access and runtime updates.
06		DeviceManagement	Manages device registration, authentication, metadata, and whitelisting. Ensures secure device integration with other modules.
07		FaceId	Provides face biometric storage and verification services, integrating with attendance and device modules for identity checks.
08		NotificationService	Orchestrates in-app, push, and email notifications, including templates, delivery channels, and read status tracking.
09		ScheduleManagement	Manages courses, class sections, schedules, rooms, and enrollments, integrating with attendance and notifications.
10		UserManagement	Handles user accounts, authentication, roles, permissions, and user requests, integrating with other modules for security and workflows.

Table 1.2.3 Package Descriptions

2. Database Design

2.1. SQL Database Diagram

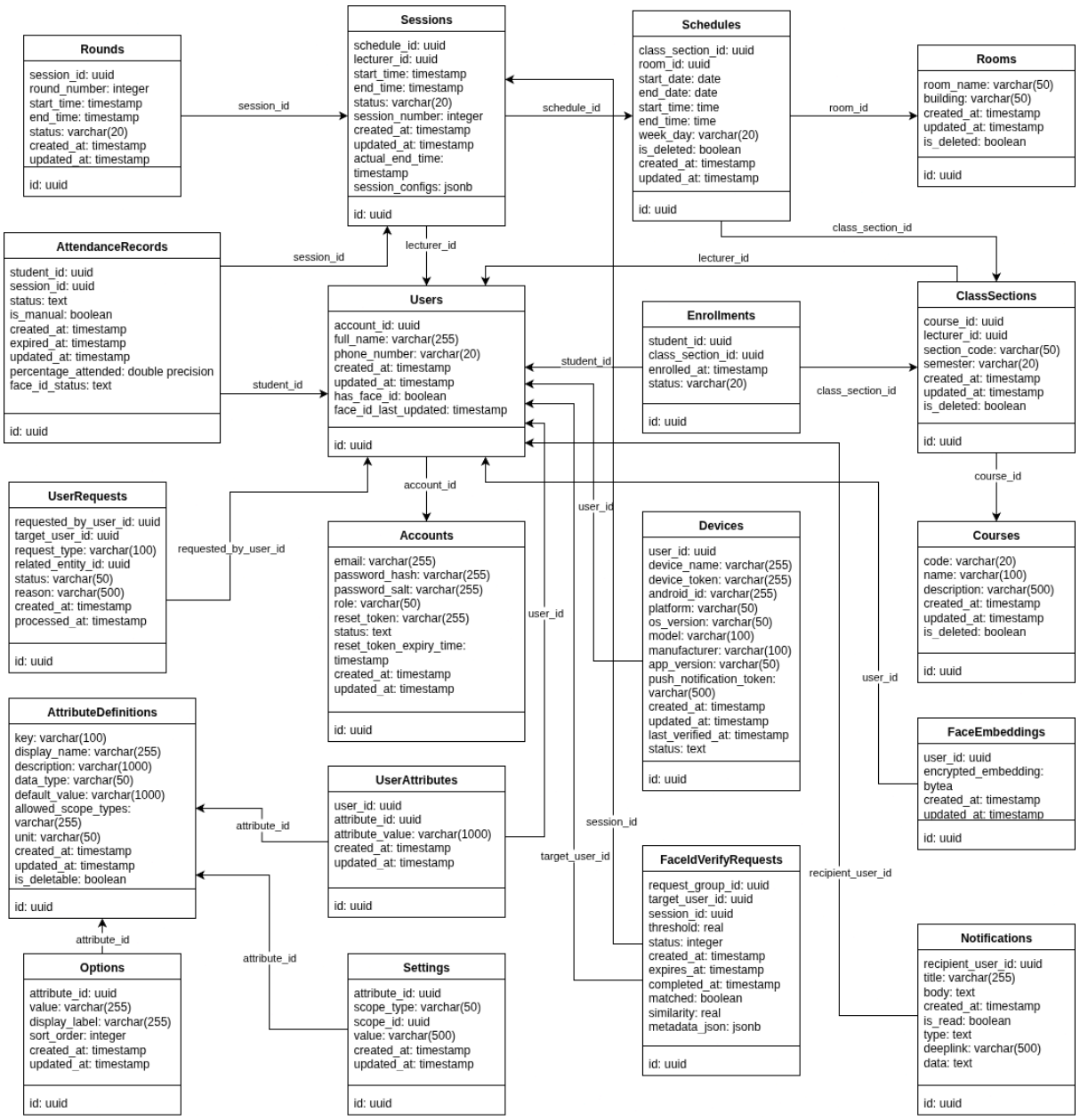


Figure 2.1 Database Design

Table Description

2.1.1 User

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	account_id	uuid	Foreign key, Not null	References Account
3	full_name	varchar(255)	Not null	User's full name
4	phone_number	varchar(255)		User's phone number
5	has_face_id	boolean	Default: false	Whether face ID data exists
6	face_id_last_updated	timestamp	Nullable	Last update time of face ID

Table 2.1.1 User Table Description

2.1.2 Account

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	email	varchar(255)	Unique, Not null	Login email
3	password_hash	varchar(128)	Not null	Hashed password
4	password_salt	varchar(64)	Not null	Salt for password hashing
5	role	enum	Not null	Admin / Student / Lecturer
6	reset_token	varchar(255)	Nullable	Token for password reset
7	reset_token_expiry_time	timestamp	Nullable	Expiration time of reset token
8	status	enum	Default: Active	Account status

Table 2.1.2 Account Table Description

2.1.3 User Request

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	requested_by_user_id	uuid	Not null	Requester user ID

3	target_user_id	uuid	Not null	Target user ID
4	request_type	enum	Not null	Request type (e.g., AbsentReport)
5	related_entity_id	uuid	Not null	Related entity ID
6	status	enum	Default: Pending	Request status
7	reason	varchar(1000)	Nullable	Reason for the request
8	processed_at	timestamp	Nullable	Time when request processed

Table 2.1.3 User Request Table Description

2.1.4 Device

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	user_id	uuid	Foreign key, Not null	References User
3	device_name	varchar(255)	Not null	Name of device
4	device_token	varchar(255)	Not null	Authentication token
5	android_id	varchar(255)	Not null	Android device ID
6	platform	varchar(50)	Nullable	OS platform
7	os_version	varchar(50)	Nullable	OS version
8	model	varchar(50)	Nullable	Device model
9	manufacturer	varchar(50)	Nullable	Device manufacturer
10	app_version	varchar(50)	Nullable	App version
11	push_notification_token	varchar(255)	Nullable	Token for push notifications
12	last_verified_at	timestamp	Nullable	Last token verification time
13	status	enum	Default: Active	Device status

Table 2.1.4 Device Table Description

2.1.5 Attribute Definition

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	key	varchar(100)	Unique, Not null	Attribute key
3	display_name	varchar(255)	Not null	Display name
4	description	varchar(1000)	Nullable	Description of attribute
5	data_type	enum	Not null	Data type (int/string/bool...)
6	default_value	varchar(500)	Nullable	Default value
7	allowed_scope_types	text[]	Not null	List of applicable scopes
8	unit	varchar(255)	Nullable	Unit of measurement
9	is_deletable	boolean	Default: true	Whether the attribute can be deleted

Table 2.1.5 Attribute Definition Table Description

2.1.6 Option

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	attribute_id	uuid	Foreign key, Not null	References AttributeDefinition
3	value	varchar(255)	Not null	Option value
4	display_label	varchar(255)	Not null	Display label
5	sort_order	int	Default: 0	Display order

Table 2.1.6 Option Table Description

2.1.7 Setting

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	attribute_id	uuid	Foreign key, Not null	References AttributeDefinition
3	scope_type	enum	Not null	Scope type

4	scope_id	uuid	Not null	Scope identifier
5	value	varchar(255)	Nullable	Setting value

Table 2.1.7 Setting Table Description

2.1.8 User Attribute

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	user_id	uuid	Foreign key, Not null	References User
3	attribute_id	uuid	Foreign key, Not null	References AttributeDefinition
4	attribute_value	varchar(255)	Nullable	Attribute value

Table 2.1.8 User Table Attribute Description

2.1.9 Course

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	code	varchar(50)	Not null	Course code
3	name	varchar(255)	Not null	Course name
4	description	varchar(1000)	Nullable	Course description
5	is_deleted	boolean	Default: false	Soft delete flag

Table 2.1.9 Course Table Description

2.1.10 Class Section

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	course_id	uuid	Foreign key, Not null	References Course
3	lecturer_id	uuid	Nullable	Lecturer assigned
4	section_code	varchar(50)	Not null	Section code
5	semester	varchar(4)	Not null	Semester (e.g., "2024")

6	is_deleted	boolean	Default: false	Soft delete flag

Table 2.1.10 Class Section Table Description

2.1.11 Enrollment

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	student_id	uuid	Not null	Student ID
3	class_section_id	uuid	Foreign key, Not null	References ClassSection
4	enrolled_at	timestamp	Not null	Enrollment time
5	status	enum	Default: Active	Enrollment status

Table 2.1.11 Enrollment Table Description

2.1.12 Room

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	room_name	varchar(100)	Not null	Room name
3	building	varchar(100)	Not null	Building name
4	capacity	int	Not null	Room capacity
5	is_deleted	boolean	Default: false	Soft delete flag

Table 2.1.12 Room Table Description

2.1.13 Schedule

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	class_section_id	uuid	Foreign key, Not null	References ClassSection
3	room_id	uuid	Foreign key, Not null	References Room
4	start_date	date	Not null	Start date
5	end_date	date	Not null	End date

6	start_time	time	Not null	Start time
7	end_time	time	Not null	End time
8	week_day	enum	Not null	Day of the week
9	is_deleted	boolean	Default: false	Soft delete flag

Table 2.1.13 Schedule Table Description

2.1.14 Session

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	schedule_id	uuid	Foreign key, Not null	References Schedule
3	lecturer_id	uuid	Nullable	Lecturer ID
4	start_time	timestamp	Not null	Session start time
5	end_time	timestamp	Not null	Session end time
6	status	enum	Not null	Session status
7	session_number	int	Not null	Sequence number
8	actual_end_time	timestamp	Nullable	Real end time
9	session_configs	jsonb	Not null	Configuration snapshot

Table 2.1.14 Session Table Description

2.1.15 Round

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	session_id	uuid	Foreign key, Not null	References Session
3	round_number	int	Not null	Round order
4	start_time	timestamp	Not null	Start time
5	end_time	timestamp	Not null	End time
6	status	enum	Not null	Round status

Table 2.1.15 Round Table Description

2.1.16 Attendance Record

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	student_id	uuid	Not null	Student ID
3	session_id	uuid	Foreign key, Not null	References Session
4	status	enum	Not null	Attendance status
5	is_manual	boolean	Not null	Marked manually
6	expired_at	timestamp	Not null	Expiration time
7	percentage_attended	double	Not null	Attendance percentage

Table 2.1.16 Attendance Record Table Description

2.1.17 Face Embedding

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	user_id	uuid	Foreign key, Not null	References User
3	encrypted_embedding	binary	Not null	Encrypted face embedding vector

Table 2.1.17 Face Embedding Table Description

2.1.18 Face ID Verify Request

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
3	request_group_id	uuid	Not nul	Group request indentifier
2	student_id	uuid	Foreign key, Not null	References User Student
3	lecture_id	uuid	Foreign key, Not nul	References User Lecture
4	session_id	uuid	Not nul	References Session

5	status	int	Not nul	Status of request
6	matched	boolean	Not nul	Verification request status
7	completed_at	Datetime	Not nul	Complete request time
8	similarity	double	Not nul	Similarity between embedding

Table 2.1.18 FaceI Id Verify Request Table Description

2.2 NoSQL Database

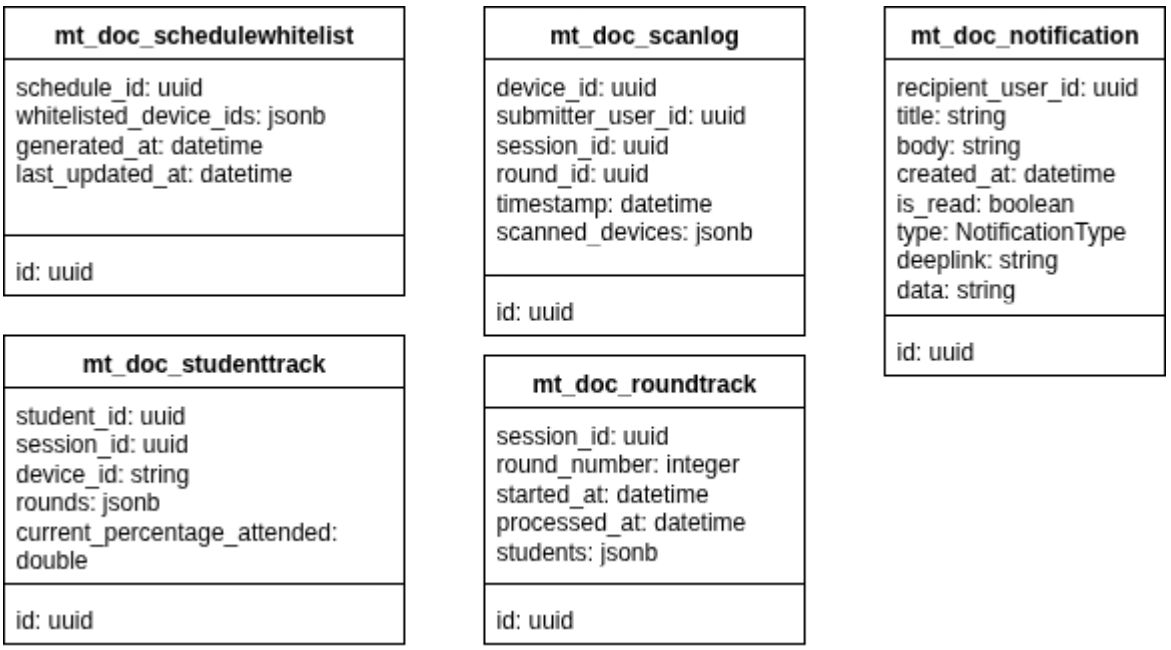


Figure 2.2 SQL Database Design

Table Description

2.2.1 Schedule Whitelist

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	schedule_id	uuid	Foreign key, Not null	References Schedule
3	whitelisted_device_ids	uuid[]	Not null	Allowed devices
4	generated_at	timestamp	Not null	Generation time
5	last_updated_at	timestamp	Nullable	Last update time

Table 2.2.1 ScheduleWhitelist Description

2.2.2 Scan Log

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	device_id	uuid	Not null	Device ID
3	submitter_user_id	uuid	Not null	Submitter ID
4	session_id	uuid	Foreign key, Not null	References Session
5	round_id	uuid	Foreign key, Not null	References Round
6	timestamp	timestamp	Not null	Scan time
7	scanned_devices	jsonb	Nullable	List of scanned devices

Table 2.2.2 Scan Log Description

2.2.3 Round Track

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	session_id	uuid	Foreign key, Not null	References Session
3	round_number	int	Not null	Round order
4	started_at	timestamp	Not null	Start time
5	processed_at	timestamp	Nullable	Processing completed time
6	students	jsonb	Nullable	Students in this round

Table 2.2.3 Round Track Description

2.2.4 Student Track

#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	student_id	uuid	Not null	Student ID
3	session_id	uuid	Foreign key, Not null	References Session
4	device_id	varchar(255)	Not null	Student's device ID
5	rounds	jsonb	Nullable	Participated rounds

6	current_percentage_attended	double	Not null	Current attendance percentage

Table 2.2.4 Student Track Description

2.2.5 Notification

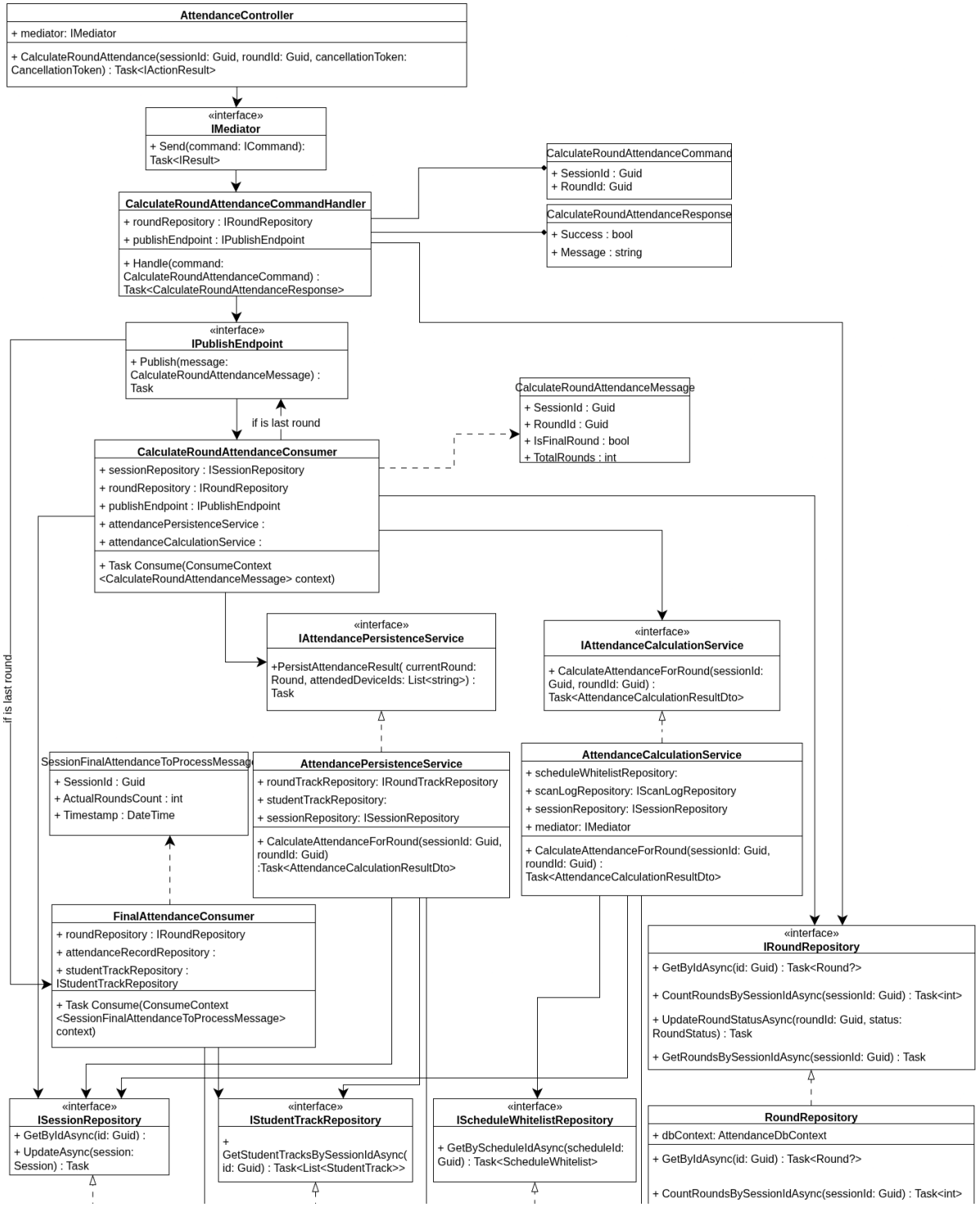
#	Field Name	Type	Constraint	Note
1	id	uuid	Primary key	Unique identifier
2	recipient_user_id	uuid	Not null	Recipient user ID
3	title	varchar(255)	Not null	Notification title
4	body	text	Not null	Notification content
5	is_read	boolean	Default: false	Read status
6	data	jsonb	Nullable	Attached data

Table 2.2.5 Notification

3. Detailed Design

3.1. Attendance Calculation

3.1.1. Class diagram



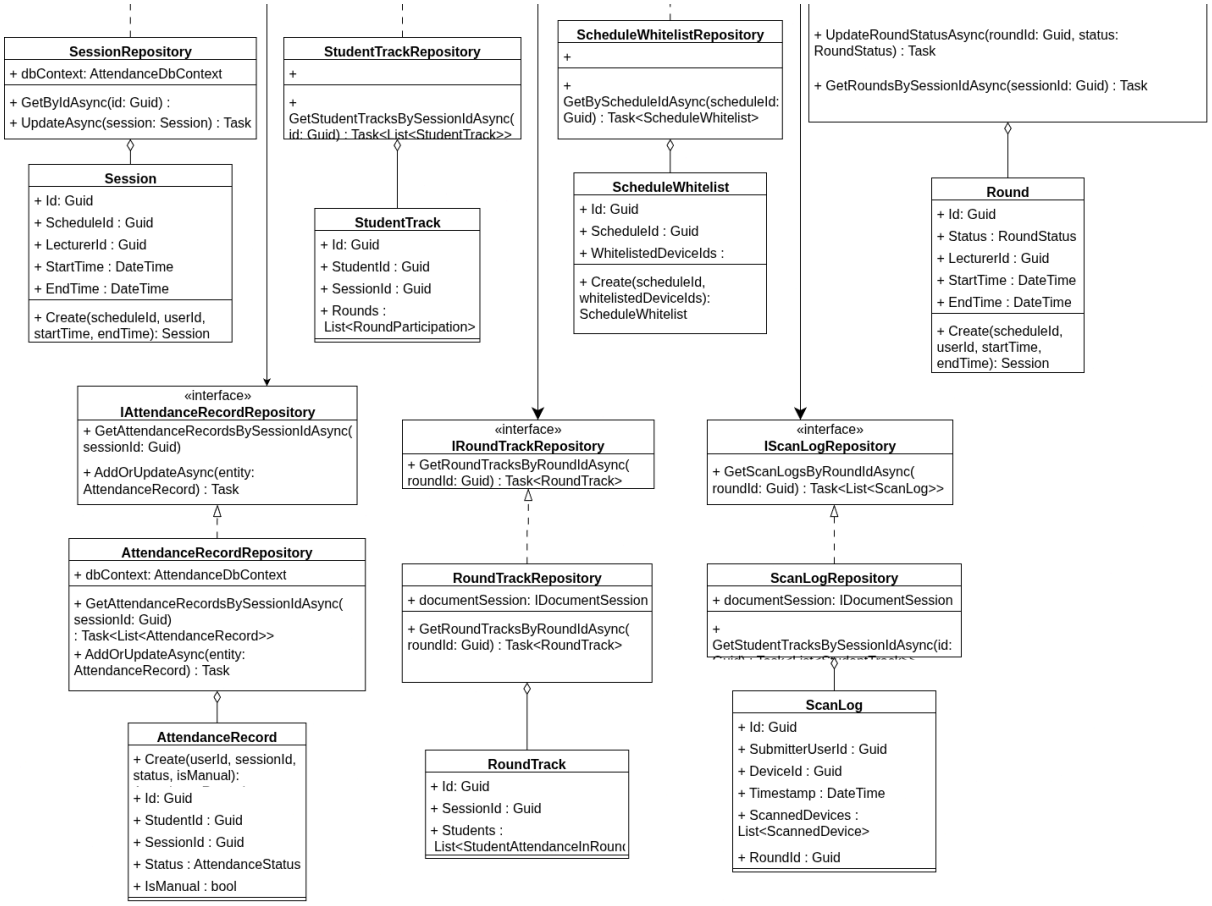


Figure 3.1.1 Attendance Calculation class diagram

3.1.2. Sequence diagram

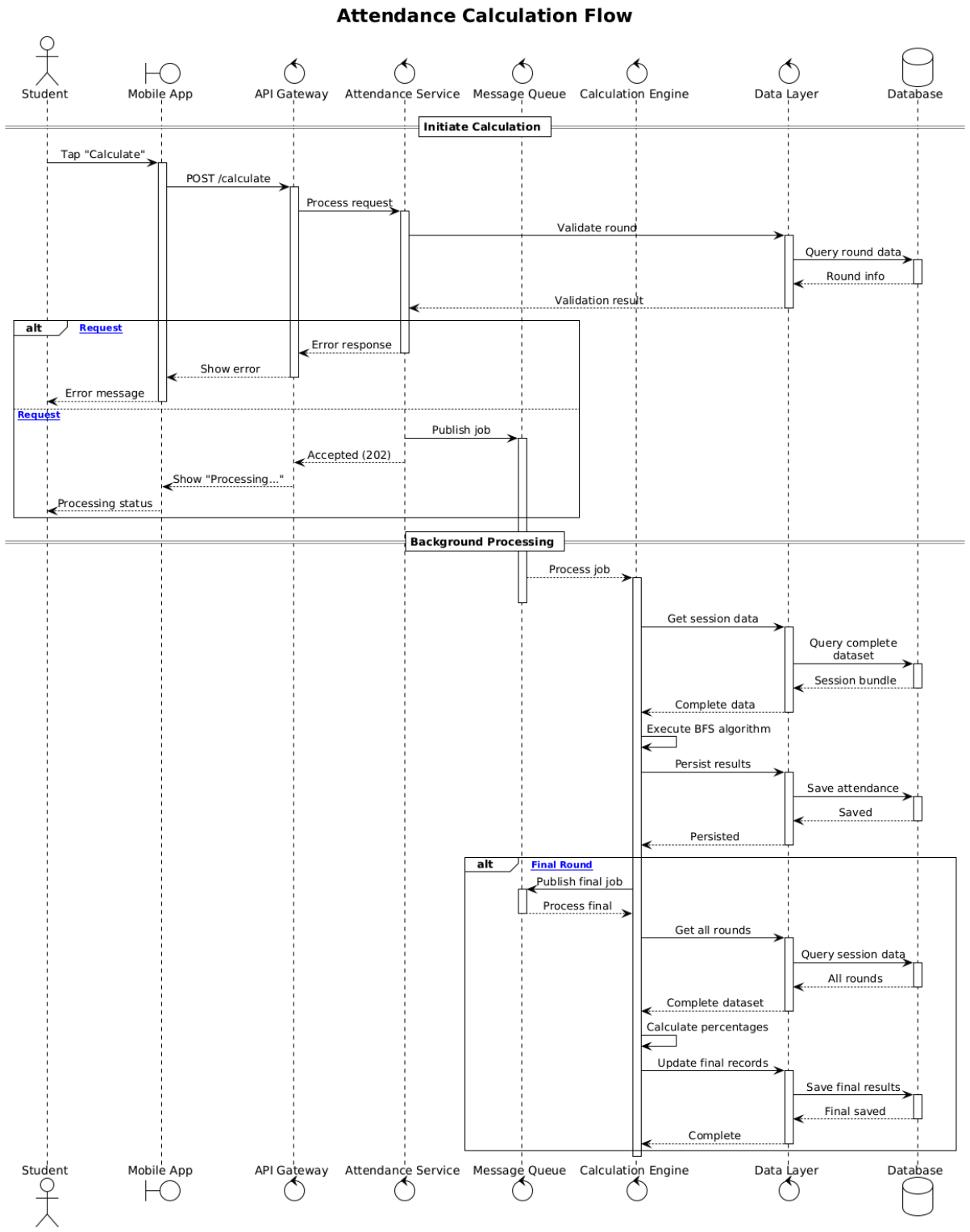


Figure 3.1.2 Attendance Calculation sequence diagram

3.2. FaceID verification

3.2.1. Class diagram

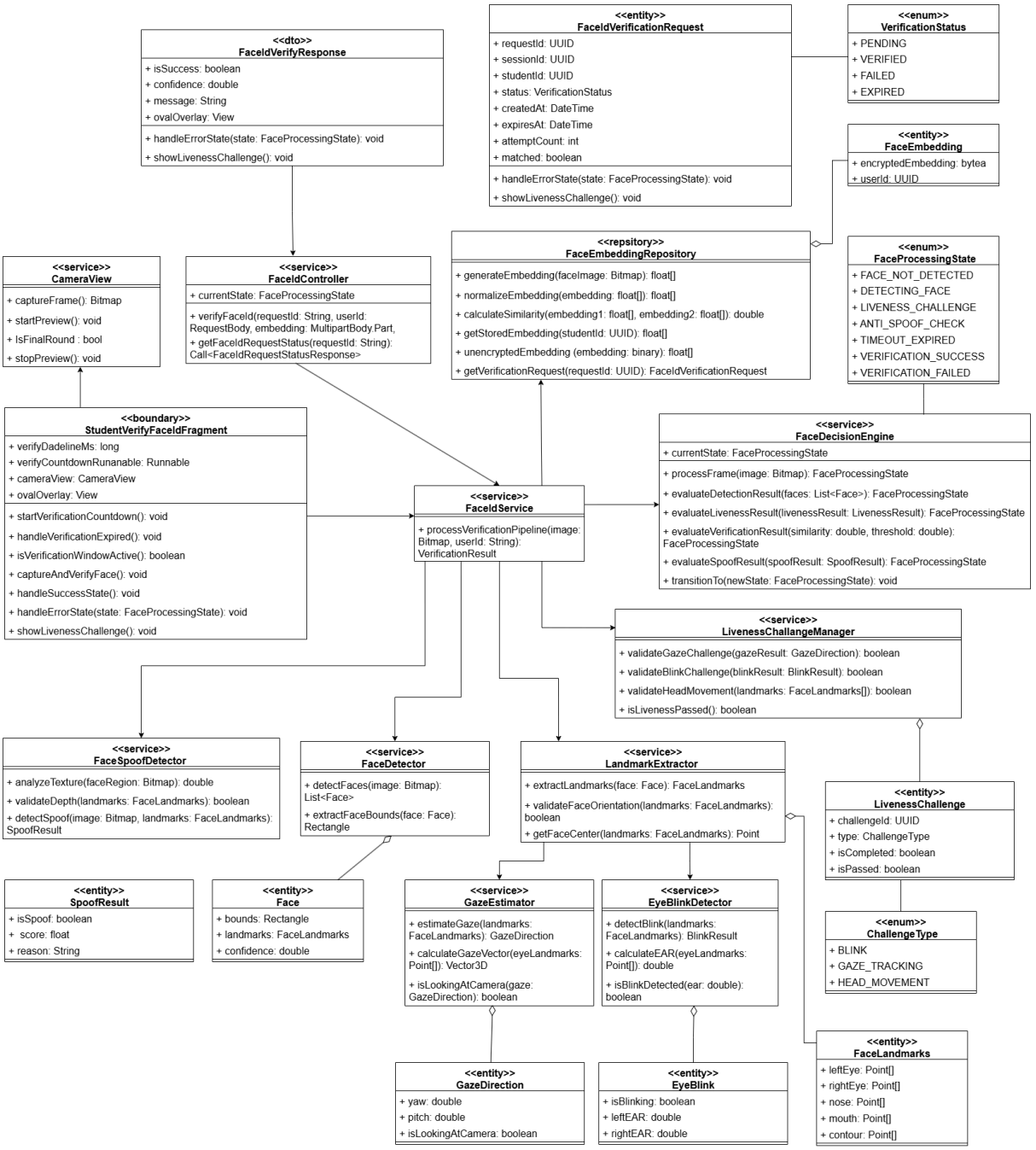


Figure 3.2.1 FaceID verification class diagram

3.2.2. Sequence diagram

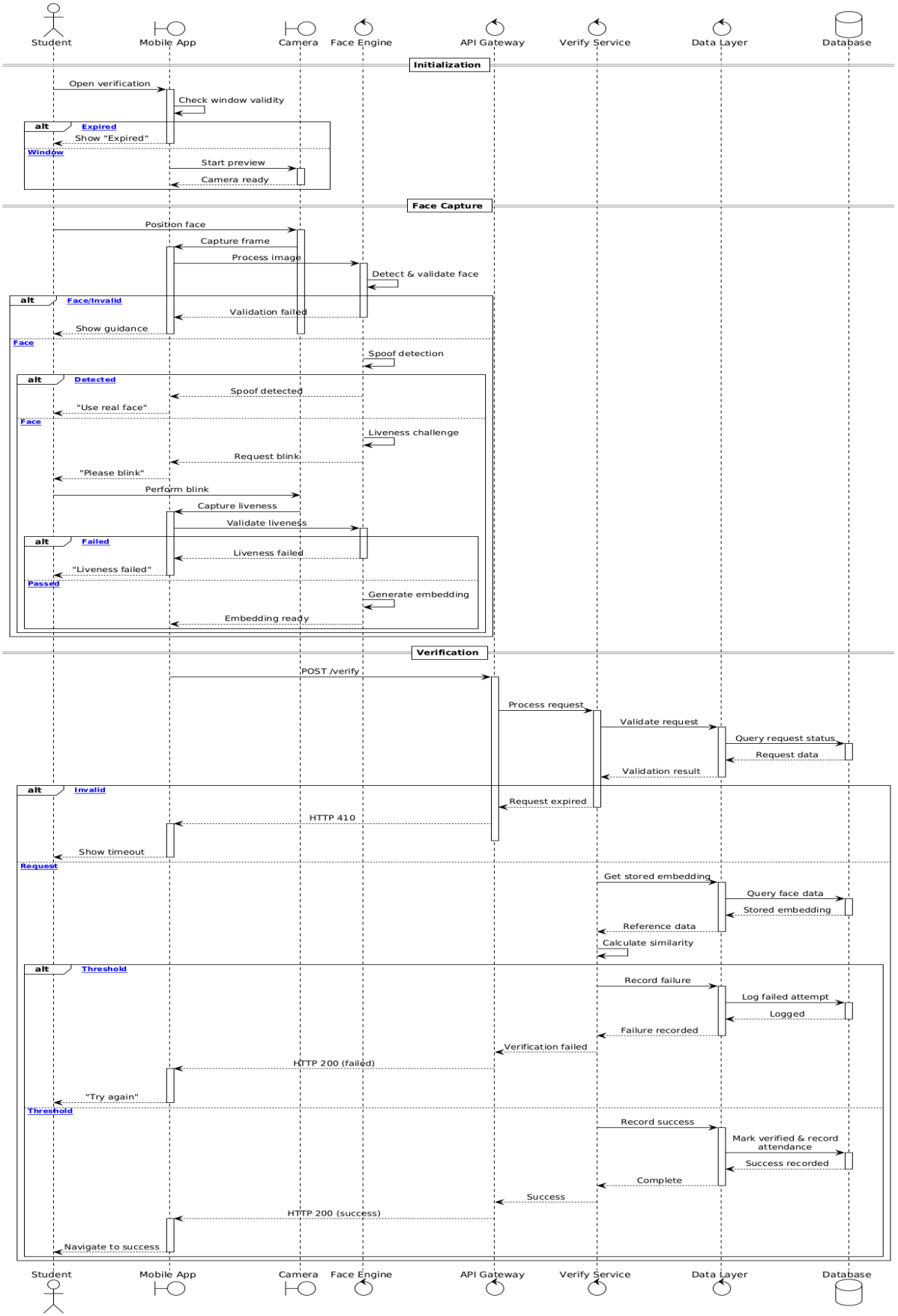


Figure 3.2.2 FaceID verification sequence diagram

4. Other Detailed Design

4.1 AI pipeline

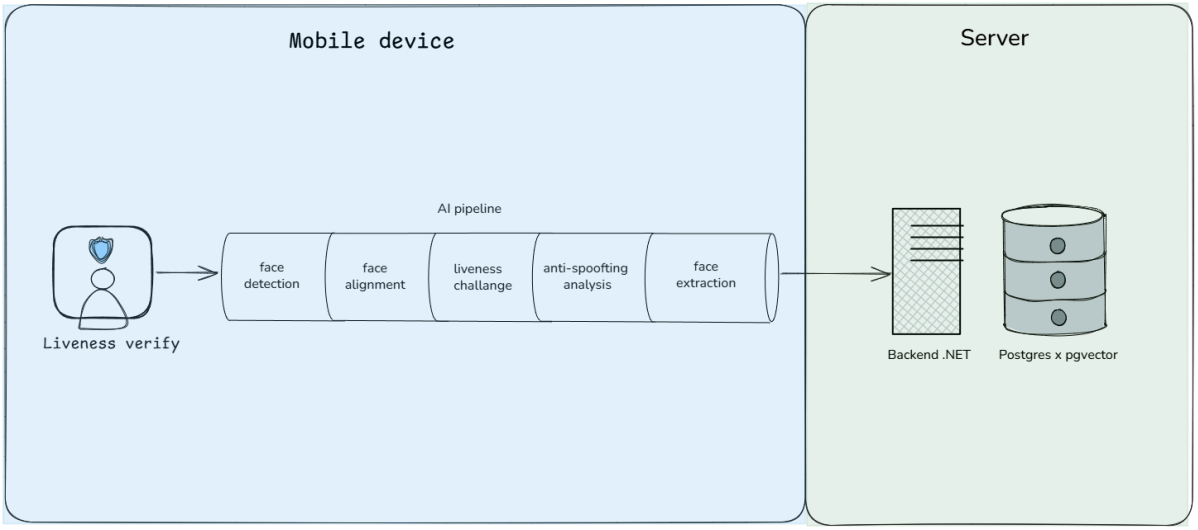


Figure 4.1 FaceID AI pipeline diagram

4.1.1 Overview

The AI face recognition pipeline implements a secure facial identification system with anti-spoofing and liveness detection capabilities. The system processes facial data through a sequential workflow that ensures authentic face verification while rejecting presentation attacks. The pipeline supports three primary operations: user registration, profile updates, and identity verification, all utilizing a standardized 512-dimensional embedding format for backend communication.

4.1.2 Face ID Verification

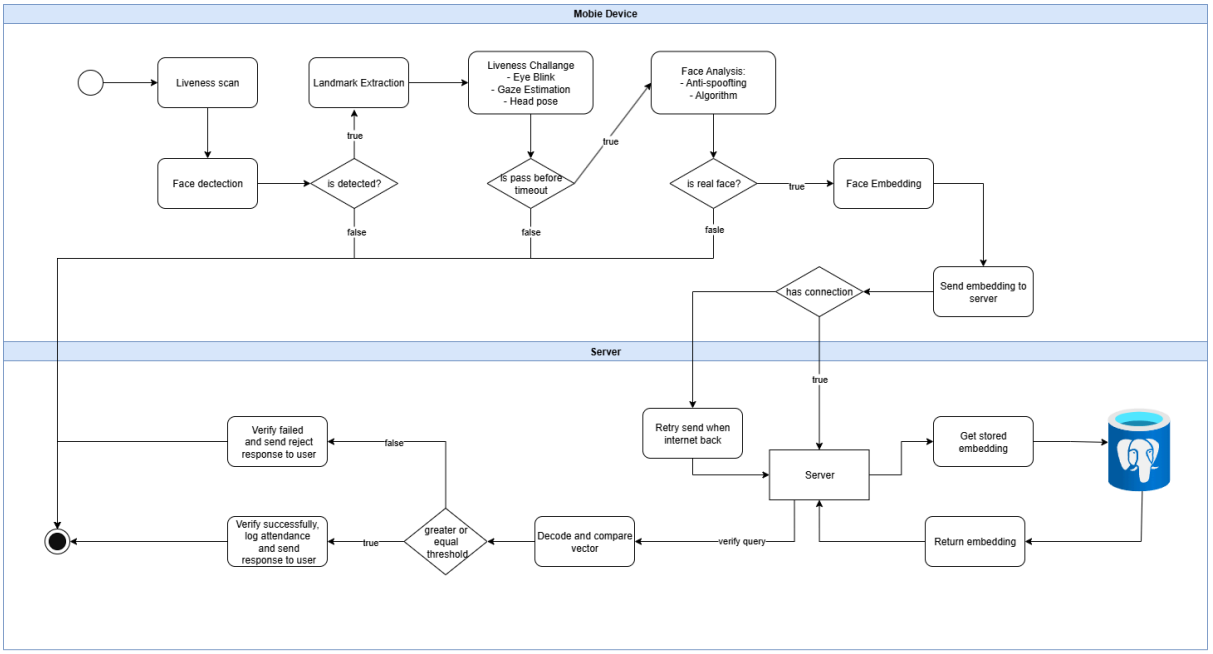


Figure 4.1.2 FaceID verification activity diagram

4.1.3 Pipeline Workflow Description

Stage 1: Face Detection and Initial Validation

The pipeline begins by capturing video frames from the camera input and processing them through a face detection algorithm. The system identifies facial regions within the frame and generates bounding boxes around detected faces. Only frames containing exactly one face proceed to the next stage - multiple faces or no face detection triggers appropriate user guidance messages. The detected face region is extracted and prepared for subsequent analysis steps.

Stage 2: Positioning and Alignment Verification

The system evaluates whether the detected face is properly positioned within the designated oval overlay area. This stage compares the face bounding box dimensions and center position against predefined tolerance thresholds. Faces that fall outside acceptable positioning parameters trigger guidance prompts directing users to adjust their position. Successfully positioned faces undergo facial landmark detection to identify key facial features including eyes, nose, and mouth for alignment purposes.

Stage 3: Anti-Spoofing Detection

The pipeline implements multi-scale anti-spoofing analysis to distinguish between genuine faces and presentation attacks. The system crops the facial region at two different scale factors (2.7x and 4.0x) and processes each through specialized deep learning models. These models analyze texture patterns, temporal variations, and other indicators of spoofing attempts including photographs, video replays, and screen displays. The system maintains temporal analysis across multiple frames to detect abnormal stability patterns and classification inconsistencies that indicate spoofing attempts.

Stage 4: Liveness Challenge Verification

Advanced liveness detection challenges users to perform specific actions proving they are physically present. The system implements two primary challenge types: blink detection and gaze direction following. Blink detection analyzes eye aspect ratios and eyelid movement patterns to verify natural blinking behavior. Gaze challenges require users to look in specific directions (typically right then left) with the system tracking eye movement through gaze estimation algorithms. Successful completion of liveness challenges enhances confidence in subsequent anti-spoofing decisions.

Stage 5: Frame Stabilization and Quality Assessment

The pipeline monitors facial positioning stability across multiple consecutive frames to ensure optimal image quality for embedding extraction. The system tracks variations in face center position and size, requiring minimal movement within defined thresholds before proceeding. A five-second analysis window collects quality scores derived from anti-spoofing confidence metrics, calculating statistical measures including mean and variance with outlier removal. Only faces meeting stability and quality criteria advance to the embedding extraction stage.

Stage 6: Feature Extraction and Embedding Generation

Stabilized facial images undergo preprocessing including resizing to 160×160 pixels and normalization to the [-1,1] value range. The processed images are fed into a FaceNet-based deep learning model that generates 512-dimensional feature vectors representing unique facial characteristics. These embeddings undergo L2 normalization to ensure consistent representation and are encoded in little-endian float32 format for backend API transmission.

Stage 7: Decision Integration and Result Processing

The final stage consolidates all pipeline outputs including anti-spoofing confidence scores, positioning compliance metrics, liveness verification status, and embedding quality assessments. The decision engine applies configurable thresholds specific to the operation type (registration, update, or verification) to determine final acceptance or rejection. Successful processing results in embedding transmission to backend services along with session metadata, while failures trigger appropriate error handling and user feedback mechanisms.

Temporal Processing and Adaptive Confidence

Throughout the pipeline, the system maintains temporal analysis capabilities that track behavior patterns across multiple frames. Post-liveness verification, the system implements adaptive confidence mechanisms that temporarily relax anti-spoofing thresholds to prevent false rejections while maintaining security integrity. This approach balances user experience with security requirements by leveraging the enhanced trust established through successful liveness challenges.

The entire pipeline operates within defined timeout parameters (30 seconds for face detection, 15 seconds for registration completion) and implements comprehensive error handling to ensure robust operation across varying environmental conditions and user behaviors

4.3 Model Description

4.3.1 Anti-spoofting model

4.3.1.1 Overview

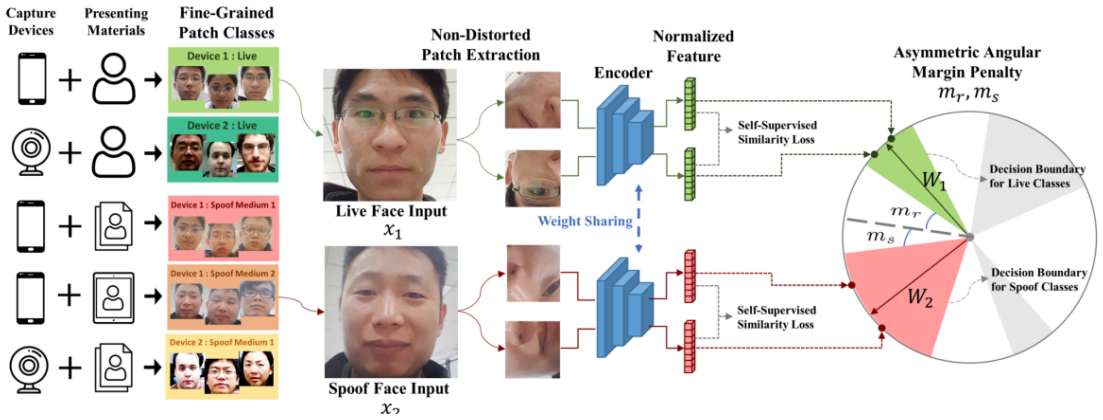


Figure 4.3.1.1.1 Patchnet learning Anti-spoofting architecture

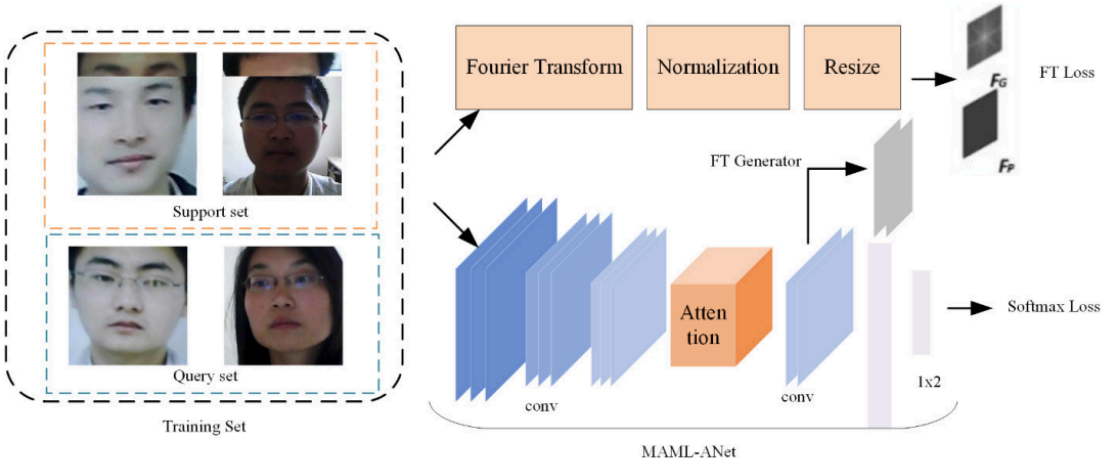


Figure 4.3.1.1.2 META learning Anti-spoofting architecture

The anti-spoofing models, **PatchSFASNet-S2.7** and **PatchSFASNet-S4.0**, are advanced solutions designed to detect presentation attacks in facial recognition systems with high accuracy and robustness, optimized for real-time inference on resource-constrained mobile devices. These models integrate fine-grained patch recognition from PatchNet with meta-learning, attention mechanisms, and Fourier spectrum supervision from the Silent Face Anti-Spoofing (SFAS) framework, achieving superior performance on the NUAA dataset (AUC >0.995, F1 >0.95, EER <0.04) compared to baselines like MiniFASNet (AUC ~0.98) and SFAS (AUC 0.99, F1 0.90). The architecture employs a dual-branch design, consisting of a main classification branch and an auxiliary Fourier supervision branch, enhanced by Model-Agnostic Meta-Learning (MAML) for rapid adaptation to unseen attack types (e.g., deepfakes, high-resolution prints).

- **PatchSFASNet-S2.7**: Optimized for moderate zoom (scale 2.7), focusing on detailed facial regions such as eyes, nose, and mouth, with approximately 3.5 million parameters. It is suitable for detecting general spoofing attacks, including print and video replay attacks.

- **PatchSFASNet-S4.0**: Enhanced for deep zoom (scale 4.0), capturing micro-textures (e.g., pixelation, moiré patterns in high-resolution prints or deepfakes), with a similar parameter count. It excels in scenarios requiring high sensitivity to subtle spoofing cues.



Figure 4.3.1.1.3 Sample Face Anti-Spoofing Dataset from NUAA



Figure 4.3.1.1.4 Sample Real Image - Patch Extraction

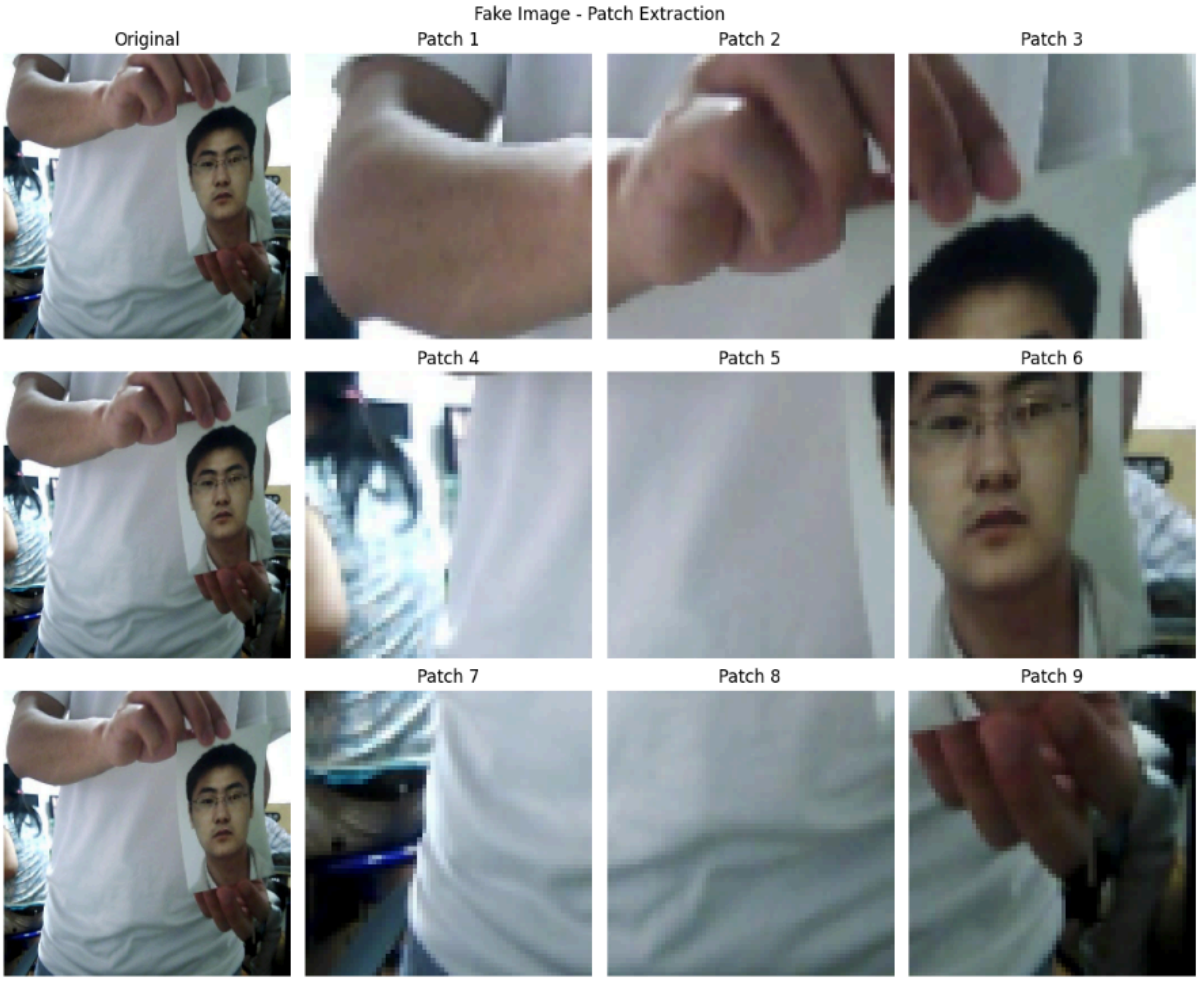


Figure 4.3.1.1.5 Sample Fake Image - Patch Extraction



Figure 4.3.1.1.6 Thermodynamic diagram example of attention mechanism

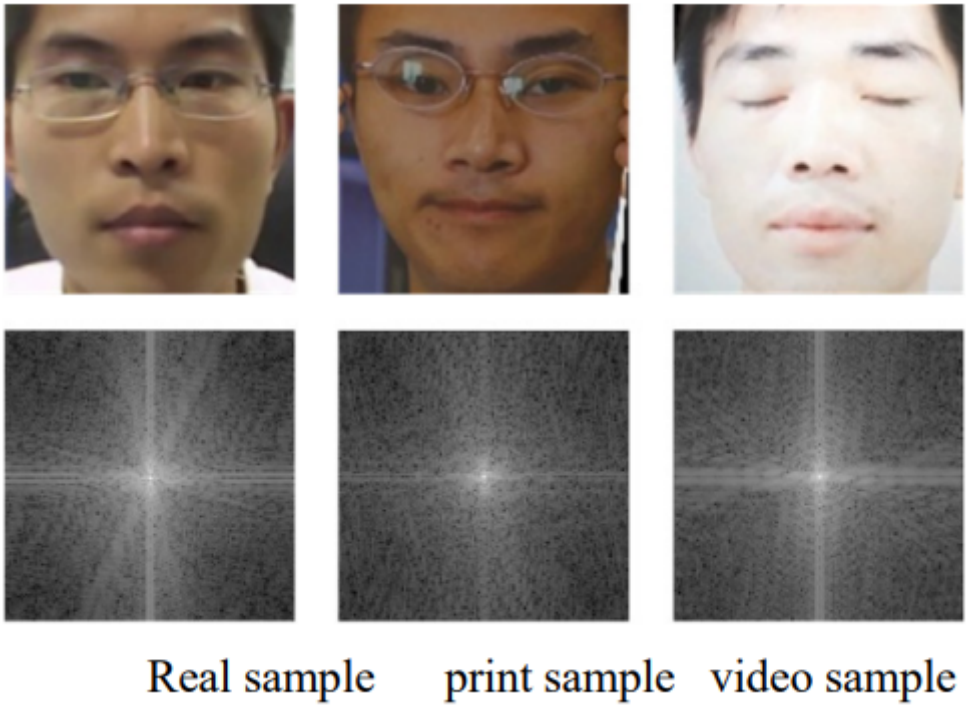


Figure 4.3.1.1.7 Fourier spectrum of face image(Top: input image, bottom: Fourier spectrum)

4.3.1.2 Architecture

The PatchSFASNet framework combines the lightweight EfficientFormerV2 backbone (from PatchNet) with advanced components from SFAS, creating a robust and efficient anti-spoofing system.

Main Classification Branch:

- Input Layer: Accepts RGB facial images of size 224×224 pixels, preprocessed at scales 2.7 or 4.0 during training to enhance data diversity and robustness.
- Convolutional Backbone: Utilizes EfficientFormerV2 (pretrained, ~3.5 million parameters), a Transformer-based architecture optimized for fine-grained patch recognition, capturing local spoofing cues such as texture artifacts.
- Attention Module: Implements a channel-wise attention mechanism (inspired by SFAS) to focus on critical facial regions (eyes, nose, mouth), reducing interference from background noise.
- Global Average Pooling: Reduces spatial dimensions while preserving feature information for efficient processing.
- Fully Connected Layer: Projects features into a 128-dimensional embedding space.
- Output Layer: Produces a binary probability distribution (live vs. spoof) using an asymmetric margin-based softmax loss (from PatchNet) to enhance generalization across diverse attack types.

Auxiliary Fourier Supervision Branch (Training Only):

- Input: 128-channel feature maps extracted from the main branch.
- Architecture: Computes Fast Fourier Transform (FFT) of input images and features, supervised by a Mean Squared Error (MSE) loss to capture frequency domain differences (e.g., high-frequency patterns at 45°/135° in live faces vs. horizontal/vertical in spoof faces).

- Purpose: Enhances feature learning by exploiting frequency domain cues, improving classification accuracy for subtle spoofing artifacts.

Meta-Learning (MAML):

- Framework: Model-Agnostic Meta-Learning (MAML) wraps the entire model, enabling few-shot adaptation (2-way 2-shot) to new attack types with minimal samples.
- Training: Employs inner and outer optimization loops to train on related tasks (e.g., varying lighting conditions, print distortions), ensuring robustness to unseen scenarios.
- Encoder:

The encoder leverages EfficientFormerV2's Transformer-based layers to extract hierarchical features, critical for distinguishing live faces from spoofing attempts such as print, replay, or deepfake attacks.

Loss Function

The models are trained using a combined loss function, integrating classification and Fourier spectrum supervision:

- Classification Loss: Cross-Entropy (CE) loss with asymmetric margin-based softmax (from PatchNet) for binary classification (live vs. spoof), promoting robust feature clustering in the embedding space.
- Fourier Loss: Mean Squared Error (MSE) loss on Fourier spectra (from SFAS), weighted by a factor of 0.3 to balance supervision signals.
- Triplet Loss: Additionally, a Triplet Loss is incorporated (inspired by domain generalization FAS literature) to further enhance the embedding space by enforcing intra-class compactness and inter-class separation, particularly for live and spoof classes. (See LaTeX section below for formula.)

Combined Loss Function

$$L = L_{\text{CE}} + 0.3 \cdot L_{\text{FT}} + \lambda \cdot L_{\text{Triplet}}$$

Cross-Entropy Loss with Asymmetric Margin:

$$L_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^{N} \log \left(\frac{e^{\cos(\theta_{y_i}) - m_{y_i}}}{\sum_{j=1}^{C} e^{\cos(\theta_j)}} \right)$$

- θ_{y_i}: angle between feature vector and class weight for y_i
- m_{y_i}: asymmetric margin (larger for live class)
- $C = 2$: number of classes (live, spoof)
- N: batch size

Fourier Transform Loss:

$$L_{\text{FT}} = \frac{1}{N} \sum_{i=1}^{N} \|\text{FFT}(x_i) - \text{FFT}(f_i)\|_2^2$$

- $\text{FFT}(x_i)$: Fourier transform of input image
- $\text{FFT}(f_i)$: Fourier transform of feature map
- N: batch size

Triplet Loss

$$L_{\text{Triplet}} = \sum_i \max\left(\|f(x_i^a) - f(x_i^p)\|_2^2 - \|f(x_i^a) - f(x_i^n)\|_2^2 + m, 0\right)$$

- $f(x_i^a)$: embedding of anchor sample
- $f(x_i^p)$: embedding of positive sample
- $f(x_i^n)$: embedding of negative sample
- $m = 0.5$: triplet margin

Loss Weighting Factor:

- Fourier loss weight: 0.3
- Triplet loss weight: $\lambda = 0.1$

Training Configuration

- Optimizer: Adam (learning rate=1e-5, beta1=0.9, beta2=0.999).
- Learning Rate Scheduler: Constant (no decay for short training duration).
- Batch Size: 4, with gradient accumulation (steps=4) to fit low VRAM constraints (e.g., 4GB).
- Total Epochs: 10, fitting within a 24-hour training constraint on Google Colab with T4 GPU.

Input

- Type: Facial images.
- Format: RGB images.
- Size: 224×224 pixels (after scaling and resizing).
- Channels: 3 (RGB).
- Preprocessing: Multi-scale crops at 2.7 (PatchSFASNet-S2.7) or 4.0 (PatchSFASNet-S4.0) during training, with scale 1.0 used for inference to optimize speed.

Output

- Type: Probability distribution over binary classes (live vs. spoof).
- Range: 0 to 1 (softmax activation).
- Classes: Live face, Spoof (encompassing print, replay, and deepfake attacks).

Key Technical Features

- Fine-Grained Patch Recognition: Inherits PatchNet's approach to analyze local facial patches, enhancing detection of subtle spoofing cues.
- Meta-Learning Adaptation: MAML enables rapid adaptation to new attack types with few samples, improving cross-domain robustness by ~5-10% compared to baselines.
- Attention Mechanism: Focuses on critical facial regions, reducing false positives from background noise.
- Fourier Supervision: Leverages frequency domain analysis to improve discrimination of live vs. spoof faces, boosting AUC by ~0.01-0.02.
- Mobile Optimization: Approximately 3.5 million parameters, ~0.4G FLOPs, and inference time <50ms on mobile devices, exportable to ONNX/TFLite formats.

4.3.2 iTracker: Mobile Gaze Estimation Model

4.3.2.1 Overview

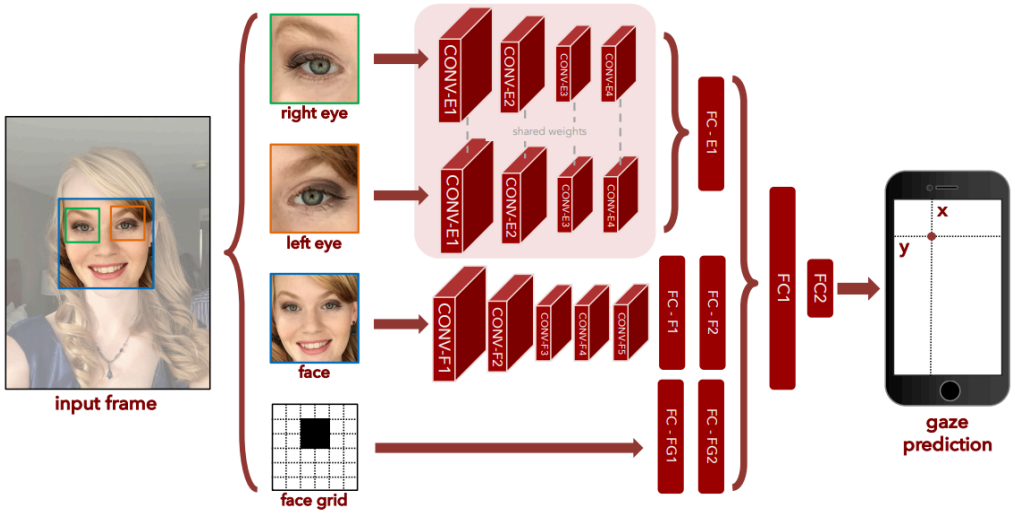


Figure 4.3.2.1 iTracker - Gaze estimation architecture

The iTracker model is a general-purpose gaze estimation CNN trained on the large-scale GazeCapture dataset, designed for mobile deployment with TensorFlow Lite. Unlike personalized approaches, iTracker achieves robust cross-user generalization without requiring individual user training, making it suitable for production applications.

Key Characteristics:

- Appearance-based CNN: End-to-end learning without hand-engineered features
- Cross-user Generalization: Trained on 1,474 subjects (2.5M frames) from the GazeCapture dataset
- Mobile Optimized: Real-time inference at 10-15fps on modern mobile devices
- Calibration Optional: Works uncalibrated with significant accuracy improvement via calibration

Performance Benchmarks (from CVPR 2016 paper):

- Uncalibrated: 1.71cm error (mobile phones), 2.53cm error (tablets)
- Calibrated (13 points): 1.34cm error (mobile phones), 2.12cm error (tablets)
- Frame Rate: 10-15fps on iPhone 6s with model compression

4.3.2.2 Architecture

Multi-Input CNN Architecture (from iTracker paper):

Full Model Inputs:

- Left Eye: 224×224×3 RGB crop
- Right Eye: 224×224×3 RGB crop
- Face: 224×224×3 RGB crop
- Face Grid: 25×25×1 binary mask indicating head location/size in frame

Compressed Model Inputs (for mobile deployment):

- Left/Right Eyes: 80×80×3 RGB crops (tighter facial landmark detection)

- Face: 224×224×3 RGB crop
- Face Grid: 25×25×1 binary mask

Encoder

Preprocessing:

- Face detection and cropping of left eye, right eye, and full face regions
- Face grid generation: 25×25 binary mask indicating normalized head position
- Image normalization to standard CNN input format
- Unified coordinate system: predicts gaze relative to camera position in centimeters

Feature Extraction:

- Separate CNN branches process each input modality
- Shared weights between left/right eye branches for parameter efficiency
- AlexNet-inspired layer dimensions with mobile optimization

Loss Function

Training Objective: Euclidean loss (L2/MSE) for 2D gaze coordinate regression:

$$L_{\text{euclidean}} = \|y_{\text{pred}} - y_{\text{true}}\|_2$$

where $y_{\text{pred}}, y_{\text{true}} \in \mathbb{R}^2$ represent predicted and ground truth coordinates in camera-relative space (centimeters).

Training Configuration:

- Framework: Caffe implementation
- Iterations: 150,000 with batch size 256
- Learning Rate: 0.001 → 0.0001 (reduced at 75k iterations)
- Optimization: Momentum 0.9, weight decay 0.0005

Calibration System

Optional Calibration using Support Vector Regression (SVR):

- Method: Extract features from FC1 layer, train per-user SVR mapping
- Calibration Points: 4, 5, 9, or 13 fixed screen locations
- Performance Improvement: 13-point calibration reduces error to 1.34cm (phones), 2.12cm (tablets)
- Implementation: Single SVR for joint (x,y) prediction or separate regressors

Input/Output Specifications

Input:

- Eye Images: 224×224×3 RGB (full model) or 80×80×3 RGB (compressed)
- Face Image: 224×224×3 RGB
- Face Grid: 25×25×1 binary mask
- Coordinate System: Camera-relative measurements

Output:

- Format: 2D coordinates (x,y) in centimeters relative to camera position
- Range: Typically ±20cm horizontally, ±25cm vertically based on device geometry
- Post-processing: Transform to screen coordinates using device-specific camera placement

Mobile Deployment Optimization

Model Compression (using dark knowledge distillation):

- Technique: Combine ground truth loss + teacher model predictions + feature matching
- Performance: ~0.05s inference time on iPhone 6s
- Memory: Reduced model size while maintaining accuracy
- Integration: Compatible with Apple's face detection pipeline for 10-15fps overall

Key Advantages Over Personalized Approaches

1. No User Training Required: Works out-of-box for new users
2. Large-Scale Training: Leverages diverse 1,474-subject dataset for robustness
3. Cross-Dataset Generalization: Features transfer well to other gaze datasets
4. Production Ready: Suitable for commercial deployment without per-user setup

Limitations

- Calibration Beneficial: While functional uncalibrated, calibration significantly improves accuracy
- Lighting/Pose Sensitivity: Performance degrades under extreme conditions despite training diversity
- Device Dependency: Requires accurate camera position measurements for coordinate transformation

4.3.3 FaceNet Model Description

4.3.3.1 Overview

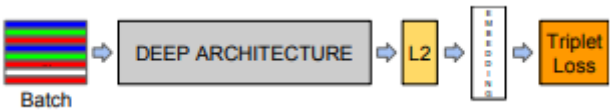


Figure 4.3.3.1 FaceNet model structure

FaceNet is a deep learning model designed for face verification, recognition, and clustering. It learns a direct mapping of face images to a compact Euclidean space, where the squared L2 distance between embeddings corresponds to a measure of face similarity. This allows for efficient and scalable face recognition using standard techniques like thresholding, k-NN, and k-means clustering on the generated embeddings.

The model is trained using a triplet loss function, which minimizes the distance between an anchor and a positive sample (same identity) while maximizing the distance between the anchor and a negative sample (different identity). While the original paper achieved state-of-the-art performance with 128-dimensional embeddings, the provided repository also contains models that produce 512-dimensional embeddings. The model has demonstrated exceptional performance on benchmarks like Labeled Faces in the Wild (LFW) and YouTube Faces DB.

4.3.3.2 Architecture

The FaceNet architecture is based on a deep convolutional network (CNN). The models in the repository primarily use an Inception ResNet v1 architecture, which combines the strengths of the Inception architecture with residual connections.

- Input Layer: The model typically takes a 160x160 pixel face image as input. This image is pre-whitened (standardized) before being passed to the network.
- Convolutional Layers: A deep stack of convolutional layers, including Inception modules and residual connections, extracts hierarchical features from the face image.

- Residual Connections: These are integral to the Inception ResNet v1 architecture, improving training efficiency and enabling deeper networks by mitigating the vanishing gradient problem.
- L2 Normalization: The output embedding is L2-normalized to ensure all embeddings lie on a hypersphere of radius 1. This is a critical step for using squared L2 distance as the similarity metric.
- Embedding Layer: The final layer is a fully connected layer that outputs a 128-dimensional or 512-dimensional vector, which represents the face embedding.
- Output Layer: There is no traditional output layer for classification. The final output is the embedding vector itself, which is then used for distance comparisons.

Encoder

The FaceNet encoder transforms a facial image into a compact Euclidean embedding. This process ensures that faces of the same person are mapped to nearby points in the embedding space, while faces of different people are mapped far apart.

- Image Preprocessing: The input image is first processed by a face alignment method like MTCNN to detect and crop the face. The cropped image is then resized to 160x160 pixels. Finally, the image is pre-whitened by subtracting the mean and dividing by the standard deviation across each channel. It is not normalized to.
- Feature Extraction: The preprocessed image is passed through the deep Inception ResNet v1 network to extract discriminative facial features.
- Embedding Generation: The features are projected by a fully connected layer to produce a compact 128-D or 512-D embedding vector. The embedding is then L2-normalized.

Objective Function

FaceNet is trained using a triplet loss function designed to enforce a margin between pairs of faces from the same identity and pairs of faces from different identities. The objective is to satisfy the triplet constraint for all possible triplets of anchor, positive, and negative samples.



Figure 4.3.3.1 FaceNet model triplet loss

The loss function is defined as:

$$L = \sum_{i=1}^{N} \left[\|f(\mathbf{x}_i^a) - f(\mathbf{x}_i^p)\|_2^2 - \|f(\mathbf{x}_i^a) - f(\mathbf{x}_i^n)\|_2^2 + \alpha \right]_+$$

where $f(\mathbf{x})$ is the embedding of image $\mathbf{x}$

$\mathbf{x}_i^a$ is the anchor image (a specific person's face)

$\mathbf{x}_i^p$ is the positive image (different image of the same person)

$\mathbf{x}_i^n$ is the negative image (different person)

α is the margin enforced between positive and negative pairs

$[z]_+ = \max(z, 0)$ ensures loss is non-zero only when margin is not met

4.3.4 Face Detection MediaPipe

4.3.4.1 Overview

BlazeFace is a lightweight face detection model that detects faces in images or video streams, even in real-time with low-latency, on mobile devices. It is optimized for on-device performance, which is key for your project, especially if it's designed to run on mobile platforms.

Key Features:

- Speed: BlazeFace is designed to be fast, making it ideal for real-time face detection on mobile devices and other low-power devices.
- Accuracy: It can detect faces in various poses and lighting conditions, with high precision.
- Robustness: The model is resilient to different facial orientations, including those at various yaw, pitch, and roll angles.
- Scalability: It is capable of detecting multiple faces simultaneously in a single frame.

4.3.4.2 Architecture:

BlazeFace is based on a lightweight CNN architecture with mobile-optimized layers. The model utilizes the Single Shot Multibox Detector (SSD) framework for object detection, which helps detect faces in real-time.

The model outputs the bounding boxes around faces, which are then passed to the Face Landmark Detection module.

Input:

- Face Image: Input images are typically resized to 320x320 pixels to fit the model's expected input size.
- Preprocessing: The input image is normalized to fit the model's requirements.

Output:

- Bounding Box Coordinates: For each detected face, BlazeFace returns the bounding box coordinates (x_min, y_min, x_max, y_max) in the image, denoting the location of the face.
- Confidence Score: A score indicating the likelihood that a detected face is valid.

Performance:

- Inference Time: BlazeFace achieves real-time performance on mobile devices with ≤ 50 ms per frame, enabling smooth interaction for applications requiring low-latency face detection.

4.3.5 Face Landmark MediaPipe

4.3.5.1 Overview

MediaPipe Face Landmarker performs detailed facial landmark detection using a regression-based neural network. It processes face regions detected by BlazeFace to extract precise 3D coordinates of

facial features, enabling advanced applications like gaze estimation, expression analysis, and augmented reality effects.

Key Features

- High-Resolution Landmarks: Detects 468 3D facial landmarks with sub-pixel precision
- Real-time Processing: Optimized for live video processing on mobile devices
- 3D Coordinate Output: Provides depth information for enhanced spatial understanding
- Expression Analysis: Generates blendshape coefficients for facial expression recognition
- Robustness: Maintains accuracy under varying lighting, poses, and partial occlusions

4.3.5.1 Architecture

- Model Type: Regression-based convolutional neural network
- Input Processing: Processes face crops from BlazeFace detection
- Multi-task Output: Simultaneous prediction of landmarks, blendshapes, and transformation matrices
- Optimization: Mobile-friendly architecture with efficient inference

Input Specifications

- Source: Face regions detected by BlazeFace
- Image Format: RGB face crops
- Preprocessing: Face alignment, resizing, and normalization
- Resolution: Variable input size based on detected face dimensions

Output Specifications: 468 3D Facial Landmarks

- Coordinate System: 3D coordinates (x, y, z) in normalized image coordinates
- Landmark Distribution:
    - Face oval: 17 points
    - Eyebrows: 18 points (9 per eyebrow)
    - Eyes: 24 points (12 per eye)
    - Nose: 27 points
    - Mouth: 20 points
    - Face contour and additional detail points: 362 points
- Precision: Sub-pixel accuracy for landmark positioning

Blendshape Coefficients

- Expression Coefficients: 52 blendshape scores representing facial expressions
- Range: Normalized values (0.0-1.0) indicating expression intensity
- Applications: Emotion recognition, avatar animation, expression transfer

Transformation Matrices

- Face Pose Matrix: 4×4 transformation matrix for face orientation
- Applications: AR effects, face alignment, pose estimation
- Coordinate System: Camera coordinate system transformation

Performance Characteristics

- Inference Time: Real-time processing capability on mobile devices (~30ms per face)
- Accuracy: High precision with sub-pixel landmark localization
- Robustness: Maintains performance under challenging conditions:
    - Partial occlusions (glasses, masks, hands)
    - Extreme lighting variations
    - Head pose angles up to ±90 degrees
    - Multiple face orientations